Driver Drowsiness Detection

The Complete Project Guide

Setup · every folder and file · every tool and why · the theory · a line-by-line tour of the code · testing · what to learn to build it yourself

CS801 Computing Lab — Real-Time Driver Drowsiness Detection with Condition-Aware Adaptive Signal Fusion

Yagnang Acharya (262IS037) · Akash Minz (262IS002)
Department of Computer Science and Engineering, NIT Karnataka, Surathkal

Describes the code on branch **fix/audit-findings** (PR #2, stacked on PR #1), October 2026. Package version 0.2.0.

Contents

1. Read this first . . . 4

How to use this guide . . . 4

The project in one paragraph . . . 4

Where the project stands . . . 5

2. Quick start (already set up on your laptop) . . . 6

3. Full setup on a new computer . . . 7

3.1 What you need . . . 7

3.2 Step by step (Windows, PowerShell) . . . 7

3.3 Linux and macOS . . . 8

3.4 The face model . . . 8

3.5 Running it . . . 8

3.6 Troubleshooting . . . 8

4. Every folder and file . . . 10

4.1 src/drowsiness/ — the application . . . 10

4.2 tests/ . . . 11

4.3 The other folders and files . . . 11

4.4 Branches and pull requests . . . 12

5. Tools: what, why, and the alternatives . . . 13

5.1 Summary . . . 13

5.2 Python . . . 13

5.3 MediaPipe FaceLandmarker — the heart of the system . . . 14

5.4 OpenCV (opencv-contrib-python) . . . 14

5.5 NumPy . . . 15

5.6 pygame (only the mixer) . . . 15

5.7 Streamlit + pandas . . . 15

5.8 Configuration: TOML, tomllib and dataclasses . . . 15

5.9 Testing and code quality: pytest, pytest-cov, ruff . . . 16

5.10 Packaging: pyproject.toml, setuptools, the drowsiness command . . . 16

5.11 Git, GitHub, pull requests and CI . . . 16

5.12 Tools of the earlier dlib version (no longer used) . . . 16

6. The science and maths . . . 17

6.1 Facial landmarks . . . 17

6.2 Eye Aspect Ratio (EAR) . . . 17

6.3 Mouth Aspect Ratio (MAR) . . . 18

6.4 Head pose with solvePnP . . . 18

6.5 Calibration . . . 19

6.6 From single frames to behaviour over time . . . 19
6.7 Fusion . . . 20
6.8 The alert state machine . . . 20
7. How the code works, module by module . . . 21
7.1 One frame's journey . . . 21
7.2 config.py . . . 21
7.3 video.py . . . 22
7.4 landmarks.py . . . 22
7.5 signals.py . . . 22
7.6 calibration.py . . . 23
7.7 indicators.py . . . 23
7.8 fusion.py . . . 23
7.9 alerts.py . . . 23
7.10 pipeline.py . . . 23
7.11 overlay.py and audio.py . . . 24
7.12 cli.py . . . 24
7.13 dashboard.py . . . 24
7.14 Design ideas worth noticing . . . 24
8. Testing: what is proven and what is not . . . 26
8.1 Kinds of tests in this project . . . 26
8.2 What the tests do NOT prove . . . 26
8.3 Live checks done on your laptop . . . 26
9. Engineering practices used . . . 27
9.1 Git commands you will use daily . . . 27
10. Safety, privacy and the audit . . . 28
10.1 What is still not safe . . . 28
11. Other ways to build a drowsiness detector . . . 29
12. What is left: Phase 3 and Phase 4 . . . 30
12.1 Phase 3 — condition detection and adaptive fusion (CE-2, Nov 1) . . . 30
12.2 Phase 4 — evaluation . . . 30
13. What to learn (in order) . . . 31
13.1 Reliable places to learn from . . . 31
14. Build it yourself: a milestone plan . . . 32
15. Viva and review questions (with answers) . . . 33
16. Glossary . . . 34

1. Read this first

This guide explains the whole Driver Drowsiness Detection project: what it does, how to set it up, what every folder and file is for, which tools are used and **why**, what the alternatives are, the maths and theory behind it, how the code works module by module, how it is tested, and what you need to learn to build something like it yourself. It also prepares you for questions in a viva or review.

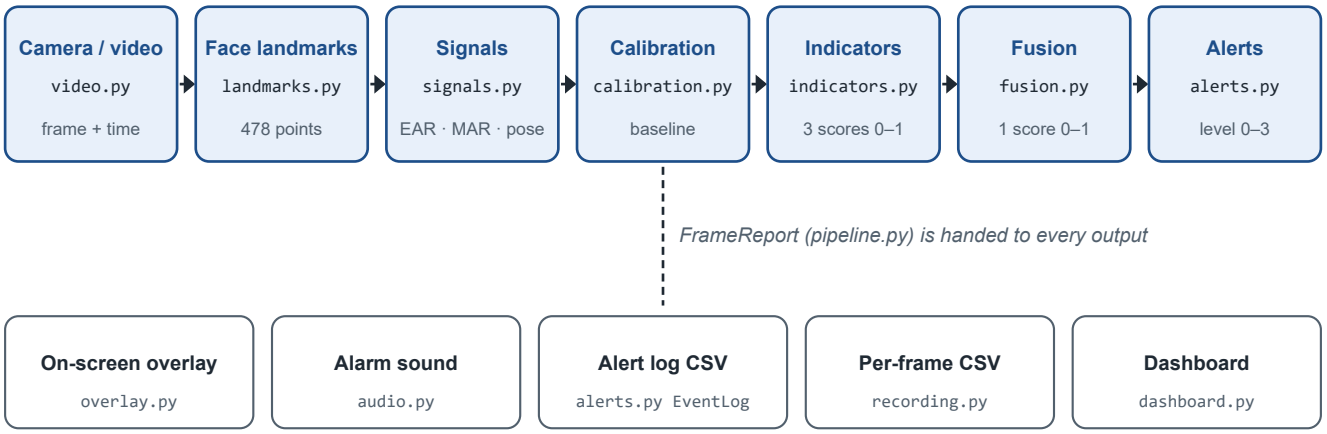
How to use this guide

If you want to...	Read
Get it running today	Sections 2 and 3
Know what each file/folder is	Section 4
Understand the tools and their alternatives	Section 5
Understand the science (EAR, MAR, head pose, PERCLOS...)	Section 6
Understand the code itself	Section 7
Know how it is tested and what is (not) proven	Section 8
Engineering practices, safety and the audit	Sections 9 and 10
Other ways to build a drowsiness detector	Section 11
What is left (Phase 3 and 4) and how to do it	Section 12
What to learn, in order, to build it yourself	Sections 13 and 14
Viva preparation and terms	Sections 15 and 16

The project in one paragraph

A webcam watches the driver. For every video frame, Google's MediaPipe finds 478 points on the face. From those points the program measures how open the eyes are (Eye Aspect Ratio, EAR), how open the mouth is (Mouth Aspect Ratio, MAR) and how the head is rotated (pitch, yaw, roll). During the first 5 seconds it learns what *this* driver's open eyes and neutral head pose look like (calibration). It then tracks, over time, how much the eyes are closed, whether the driver yawns, and whether the head nods. Those three cues are combined into one drowsiness score between 0 and 1. The score, plus some safety rules, decides an alert level: 0 normal, 1 on-screen warning, 2 alarm sound, 3 critical. Everything is shown in a live dashboard and logged to CSV files.

One frame's journey through the system



Where the project stands

Phase	Scope	State
1	Landmark pipeline; EAR, MAR, head pose	Implemented and tested; not validated on real driving data
2	Static-weight fusion, 3-level alerts, dashboard	Implemented and tested; not validated on real driving data
3	Low-light and glasses detection + adaptive fusion	Not started (CE-2, Nov 1)
4	Evaluation against the static baseline	Not started; needs labelled video
5	Polish, documentation, final report	Not started (Endsem)

Important. This is a prototype built for a course project, **not a safety device**. Its accuracy (missed drowsiness, false alarms, reaction time) has not been measured. All thresholds and weights are sensible starting values, not tuned ones. Never rely on it while driving.

2. Quick start (already set up on your laptop)

Your laptop already has Python 3.12.10 and a project environment called `.venv` with every package installed. Open a terminal in the project folder (`D:\Project\ML Project\Driver Drowsiness Detection`) and run:

```
.venv\Scripts\activate          # every new terminal; prompt starts with (.venv)
drowsiness dashboard            # opens http://127.0.0.1:8501 in your browser
```

- Turn on **Start detection**. Look straight ahead with eyes open for about 5 seconds (calibration).
- Close your eyes for about 2 seconds: the alarm sounds (Level 2). Keep them closed, or leave the camera's view for 10 seconds, to reach Level 3.
- Turn **Start detection** off to stop. You can then download the alert log.
- Stop the server with `Ctrl+C` in the terminal.

The simpler window version is `drowsiness run` (keys: `q` quit, `c` recalibrate).

> **Tip.** If PowerShell says that running scripts is disabled, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, then activate again.

3. Full setup on a new computer

These steps take a fresh Windows laptop to a working project. Linux and macOS notes follow.

3.1 What you need

Item	Why	Notes
Python 3.11 or 3.12	The language the project is written in	Install from python.org. Tick **Add python.exe to PATH**. 3.12.10 is what this laptop uses.
Git	Download the code; version control	git-scm.com. Comes with Git Bash.
A webcam	The input	Built-in laptop cameras work. Index 0 is usually the built-in one.
~1.5 GB disk	Python packages (OpenCV, MediaPipe, Streamlit, pandas, PyArrow...)	Mostly inside `.venv`.
GitHub CLI (optional)	Open pull requests from the terminal	`winget install GitHub.cli`, then `gh auth login`.

3.2 Step by step (Windows, PowerShell)

```
# 1. Get the code
git clone https://github.com/yagnangg333/Driver-Drowsiness-Detection.git
cd Driver-Drowsiness-Detection

# 2. Create an isolated environment with Python 3.12
py -3.12 -m venv .venv
.venv\Scripts\activate

# 3. Install the project and its tools, pinned to the tested versions
python -m pip install --upgrade pip
pip install -c constraints.txt -e ".[dev]"

# 4. Check everything works (96 tests, ~15 seconds)
pytest

# 5. Run it
drowsiness dashboard
```

What each install flag means

- `-e` (editable): Python uses the files in `src/` directly, so code changes apply immediately. Also required because the program finds `models/`, `assets/` and `configs/` relative to the repository folder.
- `.[dev]`: install this project (`.`) plus the optional *dev* extras: pytest, pytest-cov and ruff. Defined in `pyproject.toml`.
- `-c constraints.txt`: use exactly the versions this project was tested with (for example mediapipe 1.0.1, numpy 2.5.3, opencv-contrib-python 5.0.0.93, streamlit 1.64.0). Leave it out to get the newest versions instead.

3.3 Linux and macOS

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install -c constraints.txt -e ".[dev]"
# Linux only: system libraries that OpenCV and MediaPipe load at start-up
sudo apt-get install -y libgl1 libglib2.0-0 libegl1 libgles2
```

3.4 The face model

`models/face_landmarker.task` (3.6 MB) is committed to the repository, so nothing else has to be downloaded. If it is ever missing, `drowsiness download-model` fetches it from Google's official storage and checks its SHA-256 fingerprint so a corrupted or tampered file is rejected.

3.5 Running it

Command	What it does
`drowsiness dashboard`	Streamlit dashboard in the browser: live video, values, scores, chart, alert history. Local-only (127.0.0.1).
`drowsiness run`	OpenCV window with the overlay. `q` quits, `c` recalibrates.
`drowsiness run --record session.csv`	Same, and saves every frame's values to a CSV.
`drowsiness run --source 1`	Use camera 1, or give a video file path instead.
`drowsiness run --no-sound`	Visual alerts only.
`drowsiness process clip.mp4`	Analyse a video file without a window; writes `clip.signals.csv` and prints a summary. Used for evaluation.
`drowsiness download-model`	Download and verify the face model.
`--config my.toml`	Override any setting (copy `configs/default.toml` and edit).
`--no-calibration`	Skip calibration and use default thresholds.
`python -m drowsiness ...`	Same commands, if the `drowsiness` command is not on PATH.

3.6 Troubleshooting

Problem	Cause	Fix
`py` is not recognized	Terminal opened before Python was installed, or the launcher was not installed	Open a new terminal (restart VS Code). Or call Python by full path: `%LOCALAPPDATA%\Programs\Python\Python312\python.exe`.
`python --version` shows 3.14 from msys64	MSYS2's Python comes first on the system PATH	Use `py -3.12`, or activate `.venv` (then `python` is the right one).
Running scripts is disabled	PowerShell execution policy	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
Could not open camera 0	Another app (Zoom, Teams, Camera) holds it, or wrong index	Close the other app; try `--source 1` or the Camera index setting.
Calibration keeps restarting	Eyes looked closed, or head pitched more than 25°	Look straight at the road/screen with eyes normally open. Narrow eyes: lower `min_open_ear` in a config file.
Detection is already running in another tab	Only one dashboard session may use the camera	Stop detection in the other tab first.

Problem	Cause	Fix
No alarm sound	Sound unchecked, `--no-sound`, muted laptop, or no audio device	Check the Alarm sound box and system volume.
`libEGL.so.1` not found (Linux)	MediaPipe needs EGL/GLES at start-up	`sudo apt-get install libegl1 libgles2`
Many false alarms with glasses	Reflections lower the EAR	Recalibrate; if needed lower `ear_closed_ratio` (e.g. 0.70).

4. Every folder and file

The repository uses the standard **src layout**: importable code lives in `src/drowsiness/`, tests in `tests/`, and data the code needs (model, sound, config) in their own top-level folders. Line counts are for the current branch.

```
Driver-Drowsiness-Detection/
|-- src/drowsiness/        the application (Python package "drowsiness")
|-- tests/                 automated tests + two test images
|-- configs/               default.toml - every tunable setting
|-- models/                face_landmarker.task - MediaPipe face model (3.6 MB)
|-- assets/                alarm.wav + EAR reference pictures
|-- Docs/                  midsem report (LaTeX + PDF), midsem slides, this guide
|-- .github/workflows/     ci.yml - automatic tests on GitHub
|-- .streamlit/            config.toml - keeps the dashboard local-only
|-- pyproject.toml         package definition, dependencies, tool settings
|-- constraints.txt        exact tested versions of every dependency
|-- README.md  LICENSE  .gitignore  .gitattributes
|
|   (on your laptop only, never committed)
|-- .venv/                 the Python environment with all packages
|-- logs/                  alert logs written while the app runs
|-- drowsy_env/            old broken environment from another PC (safe to delete)
|-- .claude/               Claude Code's local launch settings
`-- src/drowsiness_detection.egg-info/   generated by pip install -e
```

4.1 src/drowsiness/ — the application

Each module has one job. They are listed in the order a frame flows through them.

File	Lines	Job	Main things inside
`__init__.py`	3	Marks the folder as a package; holds the version	`__version__ = "0.2.0"`
`__main__.py`	3	Lets you run `python -m drowsiness`	calls `cli.main()`
`config.py`	247	All settings, their defaults, validation, TOML loading	`AppConfig` and 6 section dataclasses, `load_config`, `resolve_path`, `ConfigError`
`video.py`	131	Reads frames from a camera or video file with timestamps; retries/reopens a failing camera	`VideoSource`, `parse_source`, `VideoSourceError`
`landmarks.py`	137	Runs MediaPipe FaceLandmarker; picks the driver's face; downloads the model	`FaceLandmarker`, `FaceObservation`, `download_model`
`signals.py`	136	Per-frame geometry: EAR, MAR, head pose	`eye_aspect_ratio`, `mouth_aspect_ratio`, `estimate_head_pose`, `compute_signals`
`calibration.py`	146	Learns the driver's open-eye EAR and neutral pose; rejects implausible baselines	`Calibrator`, `Baseline`
`indicators.py`	169	Time-based scores (0–1) for eyes, yawning, head tilt	`EyeClosureIndicator`, `YawnIndicator`, `HeadTiltIndicator`
`fusion.py`	59	Combines the three scores into one	`StaticFusion`, `FusionStrategy`, `Weights`
`alerts.py`	185	Turns the score into alert levels; logs level changes to CSV	`AlertManager`, `AlertLevel`, `EventLog`

File	Lines	Job	Main things inside
`pipeline.py`	167	Glues all of the above together for one frame	`DrowsinessPipeline.process()`, `FrameReport`
`overlay.py`	136	Draws eye/mouth outlines and the status panel on the frame	`draw()`
`audio.py`	52	Plays/stops the looping alarm sound	`AlarmPlayer`
`recording.py`	35	Writes one CSV row per frame (for evaluation)	`SignalRecorder`
`cli.py`	247	The `drowsiness` command and its sub-commands; fail-loud screen	`main`, `_cmd_run`, `_cmd_process`, `_cmd_dashboard`, `_fail_loud`
`dashboard.py`	309	The Streamlit web dashboard	`main`, `_run`, `_sidebar`, `_camera_lock`
`py.typed`	0	Empty marker: tells type checkers this package has type hints	—

4.2 tests/

File	What it checks
`conftest.py`	Shared test fixtures: loads the two test images; skips model tests if the model is missing
`fixtures/awake.png`, `drowsy.png`	Two real webcam frames (driver wearing glasses), cropped from screenshots of the earlier version. Awake EAR about 0.33, drowsy about 0.11.
`test_config.py`	Defaults match `configs/default.toml`; bad keys, types and ranges are rejected
`test_signals.py`	EAR/MAR formulas on hand-made points; head pose round-trips; sign conventions
`test_calibration.py`	Baseline maths, clamping, rejection of closed eyes / looking down, fallback
`test_indicators.py`	Blinks vs microsleeps, debounce, PERCLOS window, yawns, head tilt
`test_fusion_alerts.py`	Weights, all alert transitions, hysteresis, escalation, hold, face loss, microsleep rule, log retry
`test_pipeline.py`	End-to-end with the real MediaPipe model; face loss; calibration rejection; overlay; head pose on real face geometry; the `process` command
`test_video.py`	Camera dropouts, reopen, give-up; video-file timestamps
`test_audio.py`	Alarm follows the level; no crash without an audio device (uses a fake pygame)
`test_cli.py`	Dashboard is local-only by default; config errors are reported; clean download errors
`test_dashboard.py`	Dashboard renders with the disclaimer; invalid settings disable Start

4.3 The other folders and files

Path	What and why
`configs/default.toml`	Every setting with its default value and a comment. A test guarantees it always equals the defaults in `config.py`, so the documentation can never drift. Copy it to make your own config.
`models/face_landmarker.task`	MediaPipe's trained face model (float16, version 1). A `.task` file bundles a face detector, a 478-point landmark model and a blendshape model.
`assets/alarm.wav`	The alarm sound (was `music.wav` in the old version).
`assets/eye1.jpg`, `eye2.png`, `eye3.jpg`	Reference pictures of the eye landmarks/EAR from the original project; not used by the code.

Path	What and why
`Docs/`	`Docs/New/` holds the midsem report (LaTeX source with the NITK logo, and the compiled PDF), the midsem slides, and this guide.
`.github/workflows/ci.yml`	GitHub Actions: on every pull request and every push to main, installs the project on Ubuntu with Python 3.11 and 3.12, runs ruff and pytest.
`.streamlit/config.toml`	Makes Streamlit listen on 127.0.0.1 only, because the dashboard shows the webcam without a login. Also turns off usage statistics.
`pyproject.toml`	Standard Python project file: name, version, Python versions (3.11–3.12), dependencies, the `drowsiness` command, ruff and pytest settings.
`constraints.txt`	62 exact package versions from the tested environment, for reproducible installs and CI.
`README.md`	Front page on GitHub: warning, status, how it works, setup, usage, privacy, development.
`LICENSE`	MIT licence: anyone may use/modify the code with attribution, no warranty.
`.gitignore`	Files git must not track: environments, caches, `logs/`, recordings, build output.
`.gitattributes`	Line-ending normalisation and marks images, audio, models, PDFs as binary.

4.4 Branches and pull requests

Branch	Contains	Pull request
`main`	The original dlib project (3 commits)	—
`feature/mediapipe-pipeline`	Phases 1–2 rebuild + CI fix	#1 → main
`fix/audit-findings`	All audit fixes, 45 new tests	#2 → feature/mediapipe-pipeline (retargets to main after #1 is merged)

5. Tools: what, why, and the alternatives

Every tool below was chosen for a reason. For each one you get what it is, what it does in this project, why it was picked, and what you could use instead, with the trade-offs. Knowing the alternatives is what lets you defend the choices in a review.

5.1 Summary

Tool	Role here	Main alternatives
Python 3.12	Language	C++ (faster, harder), JavaScript (MediaPipe runs in the browser)
MediaPipe FaceLandmarker	478 face points + blendshapes	dlib 68 points (old version), OpenCV Facemark, face-alignment (PyTorch), OpenFace 2.0, InsightFace
OpenCV	Camera/video I/O, image ops, solvePnP, drawing	imageio / PyAV (video), Pillow (images), MediaPipe's own pose matrix
NumPy	Fast arrays, distances, matrices	Plain Python lists (slow), PyTorch tensors (overkill)
pygame (mixer)	Looping alarm sound with volume	simpleaudio, sounddevice, playsound, winsound (Windows only)
Streamlit	Web dashboard	Gradio, Flask/FastAPI + HTML/JS, Tkinter/PyQt desktop app
pandas	Tables and charts in the dashboard	Plain lists of dicts, Polars
TOML + tomllib	Configuration file	YAML (extra package), JSON (no comments), .env files
dataclasses	Typed config and results	pydantic (more validation, extra dependency), plain dicts
argparse	Command-line interface	click, typer
csv module	Alert logs and recordings	pandas, SQLite, JSON lines
pytest + pytest-cov	Tests and coverage	unittest (built in, more verbose)
ruff	Linting and formatting	flake8 + black + isort (three slower tools)
setuptools + pyproject.toml	Packaging	Poetry, uv, Hatch, PDM
constraints.txt	Pin tested versions	pip-tools, uv.lock, poetry.lock
Git + GitHub	Version control, collaboration	GitLab, Bitbucket
GitHub Actions	Automatic tests (CI)	GitLab CI, Jenkins, CircleCI
venv	Isolated environment	conda, uv, Poetry environments

5.2 Python

What: a general-purpose programming language with a huge ecosystem for computer vision and machine learning. **Why here:** MediaPipe, OpenCV and Streamlit all have first-class Python APIs, so the whole system (vision, maths, UI, tests) is one language. Speed is not a problem: MediaPipe and OpenCV do the heavy work in compiled C++, and the full pipeline measured **about 9.5 ms per frame** on this laptop (the webcam itself limits it to about 26 FPS).

Why 3.11–3.12: 3.11 added `tomllib` (reads TOML without extra packages) and the `X | None` type syntax used throughout. The project is tested on 3.11 and 3.12. Newer versions may work (MediaPipe 1.0 ships a version-independent wheel) but are untested. The old dlib version needed 3.10–3.12 because prebuilt dlib only exists for those.

Alternatives: C++ gives lower latency and is what in-car systems use, but is much slower to develop. JavaScript can run MediaPipe directly in a browser (no install), but testing and scientific tooling are weaker.

5.3 MediaPipe FaceLandmarker — the heart of the system

What: Google's on-device machine-learning library. FaceLandmarker is a pre-trained pipeline: a fast face detector (BlazeFace) finds faces, then a landmark network places **478 3D points** on each face (468 face-mesh points + 10 iris points). It can also output 52 *blendshape* scores (for example `eyeBlinkLeft`, `jawOpen`) and a head transformation matrix. It runs on the CPU in real time.

How it is used: `landmarks.py` creates it in *VIDEO* running mode, which tracks the face from frame to frame (faster and steadier than detecting from scratch every frame) and requires strictly increasing timestamps. It looks for up to 2 faces and keeps the largest one, assumed to be the driver.

Why MediaPipe (and not dlib like the old version):

- It is named in your project proposal.
- Far more points: 478 vs dlib's 68, including more eyelid points and depth (z).
- Robust to glasses and moderate head turns; the test images include glasses.
- Small model (3.6 MB vs 95 MB) and easy install (prebuilt wheels; dlib needs a C++ build on most Python versions).
- Blendshapes give a learned second opinion on blinks and yawns (useful for Phase 3).

> **Note.** MediaPipe's old `mp.solutions.face_mesh` API has been removed in current versions (1.0.1 is installed). This project uses the newer **Tasks API** (`mediapipe.tasks.python.vision.FaceLandmarker`). Many online tutorials still show the old API; their code will not run on current MediaPipe.

Alternative	Pros	Cons
dlib 68-point (our earlier version)	Classic, well documented, used in the original EAR paper	95 MB model, hard install, fewer points, weaker with glasses/pose
OpenCV Facemark (LBF)	Already part of OpenCV	68 points, less accurate, model downloaded separately
face-alignment (PyTorch)	Very accurate 2D/3D landmarks	Needs PyTorch, slow without a GPU
OpenFace 2.0	Landmarks + gaze + facial action units	C++ toolkit, heavy, awkward from Python
InsightFace / RetinaFace	Excellent detection	Only 5 or 106 points, larger models
A CNN eye-state classifier	Learns 'open/closed' directly from pixels	Needs labelled training data and training; less explainable

5.4 OpenCV (opencv-contrib-python)

What: the standard computer-vision library. **Used for:** opening the webcam and video files (`cv2.VideoCapture`), colour conversion (`cvtColor`), resizing, mirroring, drawing the overlay (`polylines`, `putText`, `rectangle`), the OpenCV window, and the head-pose maths (`cv2.solvePnP`, `cv2.Rodrigues`).

Why the contrib build: MediaPipe itself depends on `opencv-contrib-python`. Installing the plain `opencv-python` as well puts two copies of `cv2` in one environment, which conflict, so only the contrib build is listed.

Windows detail: the camera is opened with the DirectShow backend (`cv2.CAP_DSHOW`), which was measured to open the webcam in about 0.9 s instead of 3–6 s with the default Media Foundation backend.

Alternatives: imageio or PyAV for video, Pillow for images. For head pose, MediaPipe's transformation matrix could replace solvePnP; the proposal asked for solvePnP, which is also more transparent to explain.

5.5 NumPy

Arrays of numbers with fast vectorised maths. The 478 landmarks are a (478, 3) NumPy array; distances use `np.linalg.norm`; rotation matrices are 3x3 arrays. Everything in OpenCV and MediaPipe exchanges NumPy arrays, so there is no real alternative here.

5.6 pygame (only the mixer)

Plays `alarm.wav` in a loop, can change volume (Level 2 at 50%, Level 3 at 100%) and stop instantly, without blocking the video loop. **Alternatives:** `winsound` (Windows only, no volume), `playsound` (blocking, unmaintained), `simpleaudio` (no looping), `sounddevice` (lower level). pygame is the simplest cross-platform option that loops with volume control. If no audio device exists, `AlarmPlayer` logs a warning and becomes a silent no-op instead of crashing.

5.7 Streamlit + pandas

Streamlit turns a Python script into a web app: widgets (sliders, toggle), live image, metrics, progress bars, a line chart and a table, with no HTML or JavaScript. **pandas** holds the score history and alert table in the format Streamlit's chart and table expect.

How Streamlit works (important for understanding `dashboard.py`): the whole script re-runs from the top every time you touch a widget. The detection loop runs inside one of those runs; when you switch the toggle off, Streamlit stops the run by raising its own exception, and the `finally` blocks release the camera. Values that must survive re-runs go in `st.session_state`; objects shared by all browser tabs (the alarm player, the camera lock) use `@st.cache_resource`.

Alternatives: Gradio (similar, aimed at ML demos), Flask/FastAPI with a hand-written web page (full control, much more work), Tkinter or PyQt (desktop apps, no browser). `streamlit-webrtc` could read the webcam in the browser instead of on the server.

5.8 Configuration: TOML, tomllib and dataclasses

Settings live in frozen (read-only) **dataclasses** with defaults, grouped into sections: camera, landmarker, calibration, indicators, fusion, alerts. A **TOML** file can override any of them. TOML was chosen because it allows comments, is unambiguous, and Python 3.11+ reads it with the built-in `tomllib`. The loader rejects unknown keys (typos), wrong types and out-of-range values with a clear message.

Alternatives: YAML (popular but needs PyYAML and has surprising type rules), JSON (no comments), pydantic models (richer validation, extra dependency).

5.9 Testing and code quality: pytest, pytest-cov, ruff

pytest finds every `test_*.py` file and runs each `test_*` function; plain `assert` statements check results; fixtures share setup; `monkeypatch` swaps real objects (camera, sound, network) for fakes. **pytest-cov** reports which lines ran (78% overall). **ruff** is a very fast linter and formatter that replaces flake8, isort and black. Settings: line length 100, Python 3.11 target.

5.10 Packaging: pyproject.toml, setuptools, the drowsiness command

`pyproject.toml` is the modern standard file that describes a Python project. It lists the dependencies, declares that the code is in `src/`, and creates the `drowsiness` command through `[project.scripts]` (`drowsiness = "drowsiness.cli:main"`). **Alternatives:** Poetry, uv or Hatch provide the same plus lock files; `requirements.txt` alone (the old way) cannot define a command or metadata.

5.11 Git, GitHub, pull requests and CI

Git records every change as a commit. Work happens on **branches**; a **pull request** (PR) asks to merge a branch into `main` and is where changes are reviewed. **GitHub Actions** runs `.github/workflows/ci.yml` on every PR: it installs the project on a clean Ubuntu machine with Python 3.11 and 3.12, then runs ruff and pytest. The first CI run caught a real problem (MediaPipe needs `libEGL` on Linux), which a single laptop would never reveal; that is exactly what CI is for.

5.12 Tools of the earlier dlib version (no longer used)

Tool	Old role	Why it was replaced
dlib	68-point landmarks	Large model, hard install, weaker; MediaPipe is better and was in the proposal
imutils	Landmark index helpers, resizing	Not needed; NumPy/OpenCV do it directly
scipy	Euclidean distance	`np.linalg.norm` does the same without a large dependency
scikit-learn	Listed but never used	Removed

6. The science and maths

Drowsiness cannot be measured directly from a camera. Instead the system measures **visible behaviours that correlate with drowsiness**: long or frequent eye closures, yawning, and the head nodding. This section explains each measurement, derives the formulas, and shows real numbers from your test images.

6.1 Facial landmarks

A **landmark** is a numbered point at a fixed place on the face: point 33 is always the outer corner of one eye, point 1 the nose tip, point 152 the chin. MediaPipe returns 478 of them per face. Each has x and y (as fractions of the image width and height, which the code converts to pixels) and z (relative depth; negative is towards the camera). Because the numbering never changes, geometry on these points measures the face.

What	Landmark indices used (MediaPipe numbering)
Image-left eye, p1..p6	33, 160, 158, 133, 153, 144
Image-right eye, p1..p6	362, 385, 387, 263, 373, 380
Inner mouth corners	78, 308
Inner lip vertical pairs	(81, 178), (13, 14), (311, 402)
Head-pose points	1 nose tip, 152 chin, 33 and 263 eye outer corners, 61 and 291 mouth corners

"Left" and "right" in the code mean the side of the image, not the driver's own left/right, which avoids confusion when the image is mirrored.

6.2 Eye Aspect Ratio (EAR)

Introduced by Soukupová and Čech (2016) for blink detection. Six points outline the eye: p1 and p4 are the corners, p2 and p3 on the upper lid, p6 and p5 on the lower lid. EAR compares the eye's height with its width:

```
        |p2 - p6| + |p3 - p5|
EAR = ----------------------        |a - b| = distance between points a and b
            2 * |p1 - p4|
```

- Open eye: the lids are apart, so the numerator is large. Typical EAR is roughly 0.25–0.35.
- Closing eye: the lid points meet, so EAR falls towards about 0.1 (not exactly 0, because the landmark model never places the lids perfectly on top of each other).
- Dividing by the width makes EAR **scale invariant**: it does not change when you move closer to or further from the camera. The test `test_ear_is_scale_invariant` checks this.
- Averaging both eyes (`FaceSignals.ear`) reduces noise.

Worked example

Suppose the two lid distances are 3.2 px and 3.0 px and the eye is 9.5 px wide. EAR = (3.2 + 3.0) / (2 × 9.5) = 6.2 / 19 = **0.326**, a normal open eye. Your test images measure **0.33 awake** and **0.11 drowsy** (with glasses).

6.3 Mouth Aspect Ratio (MAR)

The same idea for the mouth: the average of three inner-lip openings divided by the inner mouth width. A closed mouth gives about 0–0.05 (your test images: 0.007 and 0.018). A wide yawn gives a much larger value. The yawn threshold is `yawn_mar_threshold = 0.5`, a starting value that still has to be checked on real yawns.

```
      mean( |81-178|, |13-14|, |311-402| )
MAR = -------------------------------------
                 |78 - 308|
```

6.4 Head pose with solvePnP

Goal: find the head's rotation from 2D points in the image. **Idea:** we know roughly where six points sit on an average 3D head (nose, chin, eye corners, mouth corners). The camera projects those 3D points onto the 2D image. Find the rotation and translation that make the projected model points land on the detected landmarks. That is the *Perspective-n-Point* (PnP) problem, solved by `cv2.solvePnP`.

The pinhole camera model

```
image point = K * (R * model_point + t)        (then divide by depth)

    | f  0  cx |      f  = focal length in pixels (approximated by the image width)
K = | 0  f  cy |      cx, cy = image centre
    | 0  0   1 |      R = 3x3 rotation, t = translation
```

The real focal length is unknown, so the code uses the common approximation f ≈ image width. That makes absolute angles slightly off, but calibration removes the constant offset because only *changes from the driver's neutral pose* are used.

From rotation matrix to angles

`solvePnP` returns the rotation as a vector, `cv2.Rodrigues` turns it into a 3x3 matrix R, and the code decomposes R = Rz(roll) · Ry(yaw) · Rx(pitch):

```
pitch = atan2(R[2,1], R[2,2])     > 0 : chin down (nodding)
yaw   = asin(-R[2,0])             > 0 : face turned towards image-right
roll  = atan2(R[1,0], R[0,0])     > 0 : head tilted clockwise in the image
```

Verified on real face geometry: rotating your real 3D landmarks by +10°, +20° and +30° of nod gave estimates of +10.5°, +18.8° and +26.6°. A 45° head turn (checking a mirror) changed pitch by at most about 6°, well below the 15° nod threshold. `test_head_pose_on_real_face_geometry` keeps checking this.

> **Note.** The 3D model coordinates are in arbitrary units (about 4.5 times a real face in mm). Scale only changes the translation, which is not used, never the rotation.

6.5 Calibration

People's eyes differ: one driver's open EAR may be 0.24, another's 0.36; glasses and camera height change it further. A fixed threshold (the old version used 0.25) is therefore wrong for many people. For the first 5 s the driver looks ahead with eyes open and the system takes the **median** of the signals (the median ignores the occasional blink, unlike the mean).

```
closed-eye threshold = clamp( 0.75 * open_EAR , 0.12 , 0.30 )
neutral pitch / roll = median over the calibration period
```

Plausibility check: a baseline is rejected and calibration restarts if the open EAR is below 0.20 (eyes probably already closed or drowsy), above 0.45, or the head pitch is beyond 25° (looking down). After 3 rejections, or 15 s with no face, the default threshold 0.21 is used and a warning stays on screen. Without this check, calibrating while drowsy silently made detection weaker (found in the audit).

6.6 From single frames to behaviour over time

One frame tells you almost nothing: everyone blinks 15–20 times a minute. What matters is **how long** and **how often**. All three indicators therefore use timestamps in seconds (not frame counts), so they behave the same at 10 FPS or 30 FPS.

Eye closure: closed-eye ratio + microsleep

- **Closed** = EAR below the calibrated threshold.
- **Closed-eye ratio (PERCLOS-like)** = seconds with eyes closed in the last 20 s, divided by 20. Dividing by the full window (not the elapsed time) avoids a false alarm from the very first blink.
- **Microsleep** = one continuous closure of 1.5 s or more.
- **Eye score** = the larger of ratio / 0.4 and closure_time / 1.5, capped at 1.
- **Debounce:** a closure only ends after the eyes stay open for 0.15 s, so one noisy frame in the middle cannot reset the microsleep timer.

Example: a normal blink of 0.2 s gives a closure score of 0.2 / 1.5 = 0.13, harmless. Eyes closed for 1.5 s give 1.0, a microsleep.

> **Important.** Classic **PERCLOS** (Wierwille and colleagues; Dinges and Grace, 1998) counts time with the eyelids at least 80% closed (the P80 criterion), usually over 1–3 minutes. This project's "closed" means EAR below 75% of the open value, a much looser criterion, over 20 s. So call it **PERCLOS-like** in your report and do not compare its numbers directly with published PERCLOS values.

Yawning

A yawn is counted once when MAR stays above 0.5 for at least 1.5 s (talking opens the mouth only briefly). The yawn score is the number of yawns in the last 60 s divided by 3, capped at 1.

Head tilt

The head counts as tilted when it is nodded forward more than 15° (pitch relative to calibration) or tilted sideways more than 20° (roll). The head score is the tilt duration divided by 2 s, capped at 1. Looking *up* does not count; nodding forward does.

6.7 Fusion

The three scores are combined with a weighted average (StaticFusion, the baseline for your Phase 3 comparison):

```
fused = 0.5 * eye + 0.2 * yawn + 0.3 * head        (weights are normalised to sum to 1)
```

Consequence you must understand: a weighted average caps what one cue can do. Yawning alone can reach at most 0.2, below the Level 1 threshold of 0.25, so it never alerts by itself. Head tilt alone reaches at most 0.3 (Level 1, visual only). Eyes alone reach 0.5 (Level 2). That is why a separate safety rule makes a **microsleep always raise at least Level 2**. Phase 3's adaptive fusion will change the weights when a cue is unreliable (for example eyes behind glasses or in the dark).

6.8 The alert state machine

Alert levels and what moves between them



Going down: the score must fall 0.05 below the level's threshold (hysteresis).
Critical is held for at least 5 s after its cause clears. Level 2+ plays the alarm (Level 3 at full volume).

Rule	Why it exists
Thresholds 0.25 / 0.45 / 0.65 on the fused score	Escalating response: warn early, sound only when evidence is stronger
Hysteresis 0.05	Stops the level flickering when the score hovers around a threshold
Level 2 held for 5 s becomes Level 3	A driver who does not respond to the alarm needs a stronger alert
Level 3 held at least 5 s	Prevents a critical alert from vanishing the moment the eyes flicker open
Microsleep forces at least Level 2	Removes the single-cue ceiling of the weighted average for the most dangerous cue
No face for 3 / 6 / 10 s gives Level 1 / 2 / 3	A slumped head or blocked camera is exactly when the driver cannot be watched; it must not stay silent
Monitoring stops (camera lost, crash) gives the alarm until acknowledged	Fail loudly: the driver must know they are no longer protected

A **state machine** is a system with a current state (here the level) and rules that say when to move to another state. Writing alerts this way makes every transition explicit and easy to test; `test_fusion_alerts.py` checks each rule.

7. How the code works, module by module

Read this section with the code open next to it. Snippets are short excerpts from the real files; `...` marks omitted lines.

7.1 One frame's journey

```
VideoSource            yields (frame_bgr, t)              video.py
  -> FaceLandmarker.detect(frame, t)  -> FaceObservation | None    landmarks.py
  -> compute_signals(face)            -> FaceSignals (ear, mar, pose)   signals.py
  -> Calibrator.update(...)           while calibrating           calibration.py
     or the three indicators          -> EyeState, YawnState, HeadState   indicators.py
  -> StaticFusion.fuse(scores)        -> FusionResult (score, weights)    fusion.py
  -> AlertManager.update(...)         -> level (+ AlertEvent when it changes)  alerts.py
  = FrameReport                       everything above, for one frame     pipeline.py
  -> overlay.draw / AlarmPlayer.set_level / EventLog.record / SignalRecorder.write
```

`DrowsinessPipeline.process(frame, t)` in `pipeline.py` performs the whole chain and returns a `FrameReport`. The CLI, the dashboard and the tests all just call `process` in a loop and decide what to do with the report. Keeping the logic in one place means the two user interfaces cannot behave differently.

7.2 config.py

Defines `AppConfig`, made of six frozen dataclasses. Frozen means a config cannot be changed by accident while running; to change a value you create a new one with `dataclasses.replace`, which is what the dashboard does when you move a slider.

```
@dataclass(frozen=True)
class AlertConfig:
    level1_threshold: float = 0.25
    level2_threshold: float = 0.45
    level3_threshold: float = 0.65
    ...
    def __post_init__(self) -> None:          # runs after every construction
        if not 0 < self.level1_threshold < self.level2_threshold < self.level3_threshold <= 1:
            raise ConfigError("alert thresholds must satisfy 0 < level1 < level2 < level3 <= 1")
```

- `load_config(path)`: reads TOML; `config_from_dict` rejects unknown sections/keys and wrong types (an int is accepted where a float is expected).
- `__post_init__` checks ranges, for example windows must be above 0. Before the audit, `perclos_window_s = 0` passed and then crashed with a division by zero.
- `resolve_path` turns relative paths such as `models/face_landmarker.task` into paths inside the repository (`PROJECT_ROOT`), so the app works from any working directory.

7.3 video.py

`VideoSource` is a context manager (`with VideoSource(0) as video:`) that always releases the camera, and an iterator yielding `(frame, timestamp)`.

- **Live camera:** timestamp = seconds since start (`time.monotonic`, which never jumps backwards like the wall clock can). Frames are mirrored for a selfie view.
- **Video file:** timestamp = the file's own time for that frame (`CAP_PROP_POS_MSEC`), so variable-frame-rate clips are timed correctly during evaluation; falls back to frame number ÷ FPS.
- **Robustness:** if the camera stops delivering frames, `_read_live` retries for up to 2 s and reopens the device once after 1 s; only then does it raise `VideoSourceError`.
- Frames wider than 640 px are shrunk, which keeps processing time stable.

7.4 landmarks.py

```
options = vision.FaceLandmarkerOptions(
    base_options=BaseOptions(model_asset_path=str(model_path)),
    running_mode=vision.RunningMode.VIDEO,      # tracks between frames
    num_faces=config.max_faces,                 # 2: driver + possible passenger
    output_face_blendshapes=True, ...)
...
timestamp_ms = max(int(timestamp_s * 1000), self._last_timestamp_ms + 1)  # must increase
result = self._landmarker.detect_for_video(mp.Image(...rgb...), timestamp_ms)
...
index = max(range(len(faces)), key=lambda i: _bbox_area(faces[i]))      # largest face
```

- OpenCV gives BGR colour order, MediaPipe expects RGB, hence `cvtColor(..., BGR2RGB)`.
- MediaPipe returns coordinates as fractions; they are multiplied by width/height to get pixels, giving a (478, 3) NumPy array inside a `FaceObservation`.
- `download_model` downloads to a `.part` file, verifies the SHA-256 checksum, then renames it, so a half-finished or tampered download never becomes the real model.

7.5 signals.py

```
def eye_aspect_ratio(eye: np.ndarray) -> float:
    horizontal = np.linalg.norm(eye[0] - eye[3])
    if horizontal < 1e-6:
        return 0.0                              # avoid dividing by zero
    vertical = np.linalg.norm(eye[1] - eye[5]) + np.linalg.norm(eye[2] - eye[4])
    return float(vertical / (2.0 * horizontal))
```

`estimate_head_pose` builds the camera matrix, gives `solvePnP` a starting guess (a frontal face at the distance implied by the eye span) so it converges to the physically sensible answer, then converts the rotation to pitch/yaw/roll. `compute_signals` returns all of it as a `FaceSignals` object.

7.6 calibration.py

`Calibrator.update(t, signals)` collects EAR, MAR, pitch and roll samples. When at least 5 s have passed *and* at least 30 samples exist, `_finish_attempt` computes medians, runs `_implausible` and either accepts the baseline, restarts, or (after 3 attempts) falls back to defaults. `note` holds the human-readable reason, shown on screen. `progress` (0–1) drives the calibration progress bar.

7.7 indicators.py

Three small classes, each with `update()` and `reset()` and an immutable state object they return. The interesting part is the eye indicator's rolling window:

```
self._samples.append((t, dt, closed))           # deque: fast append/pop at both ends
if closed:
    self._closed_time += dt
while self._samples and t - self._samples[0][0] > self._window_s:
    _, old_dt, old_closed = self._samples.popleft()     # forget samples older than 20 s
    if old_closed:
        self._closed_time -= old_dt
```

Keeping a running total instead of re-adding the whole window every frame makes each update cost the same regardless of window length. `dt` is capped at 0.5 s so a gap (for example while the face was lost) cannot inflate the ratio. If no closed samples remain, the total is reset to exactly 0 to remove floating-point drift (a bug the tests caught).

7.8 fusion.py

`FusionStrategy` is a **Protocol**: any class with a `name` and a `fuse(scores) -> FusionResult` method can be plugged into the pipeline. `StaticFusion` is the baseline. Phase 3's `AdaptiveFusion` will implement the same interface, so both can be run on identical inputs for a fair comparison. This is the *strategy pattern*.

7.9 alerts.py

`AlertManager.update(t, score, face_detected, microsleep)` applies the rules of section 6.8 in a fixed order: score thresholds with hysteresis, then the microsleep floor, then face-loss escalation, then Level 2 → 3 escalation, then the Level 3 hold. It returns an `AlertEvent` only when the level changes. `EventLog` keeps events in memory and appends them to `logs/alerts_<date_time>.csv`; if the file is locked (for example open in Excel) it logs a warning and writes the missed rows later instead of crashing the session.

7.10 pipeline.py

```
def process(self, frame_bgr, timestamp) -> FrameReport:
    face = self._landmarker.detect(frame_bgr, timestamp)
    signals = compute_signals(face) if face is not None else None
    calibrating = not self._calibrator.done
    if calibrating:
        self._calibrator.update(timestamp, signals)
    elif signals is not None:
        self._update_indicators(timestamp, signals, self._calibrator.result)
    fusion = self.fusion.fuse(IndicatorScores(...))
    if not calibrating:
        event = self.alerts.update(timestamp, fusion.score, face is not None,
                                   self._eye.state.microsleep)
    return FrameReport(...)
```

`FrameReport.to_row()` flattens a report into one CSV row (timestamp, EAR, MAR, angles, flags, scores, level) for recordings and evaluation. `recalibrate()` restarts calibration and clears all temporal state.

7.11 overlay.py and audio.py

`overlay.draw(frame, report)` draws eye and inner-lip outlines (red when closed or yawning), a status panel (EAR, MAR, angles, closed-eye %, three score bars) and a coloured banner for the level; during calibration a progress bar and any rejection note. `AlarmPlayer.set_level(level)` plays the looping sound at 50% for Level 2, 100% for Level 3, and stops it otherwise.

7.12 cli.py

Uses `argparse` sub-commands. Each sub-command maps to a `_cmd_*` function. Notable behaviour:

- `main()` turns expected errors (bad config, missing files, camera problems) into a clear message and exit code 1 instead of a traceback.
- `_cmd_run` is the OpenCV loop. If monitoring stops *after* it started (camera lost, unexpected error), `_fail_loud` shows a red screen and sounds the alarm until you press `q`.
- `_cmd_process` runs a video file through the pipeline, writes the per-frame CSV and prints the share of frames at each level.
- `_cmd_dashboard` starts Streamlit with `--server.address 127.0.0.1`; another address needs `--address` and logs a privacy warning.

7.13 dashboard.py

- `_sidebar` builds the settings widgets, disabled while detection runs (so a change cannot silently restart and recalibrate), and validates them into a new `AppConfig`.
- `_camera_lock()` is one lock shared by every browser tab (`@st.cache_resource`); a second tab that tries to start detection is refused.
- `_run` is the live loop: process, play alarm, record events, draw, and refresh the panel every 0.3 s and the chart every 1 s (updating them every frame would slow the video).
- If monitoring stops unexpectedly the alarm sounds until you switch detection off.
- Streamlit stops a run by raising its own exceptions; the code re-raises those untouched so the `finally` blocks always release the camera and stop the sound.

7.14 Design ideas worth noticing

Idea	Where	Benefit
Single responsibility per module	everything	Each file can be understood and tested alone
Pure functions for maths	`signals.py`	Easy to test with hand-made inputs
Immutable result objects	`FrameReport`, states, config	No hidden changes; safe to pass around
Strategy pattern	`fusion.py`	Swap static/adaptive fusion without touching the pipeline
State machine	`alerts.py`	Every transition explicit and testable
Context managers (`with`)	camera, landmarker, recorder	Resources always released, even on errors
Time-based, not frame-based	indicators, alerts	Same behaviour at any FPS
Fail loudly	alerts, cli, dashboard	Never silently stop protecting the driver

Idea	Where	Benefit
Config validated at load	`config.py`	Bad settings fail immediately with a clear message

8. Testing: what is proven and what is not

There are **96 automated tests**, all passing, covering 78% of the code lines. Run them with `pytest` (about 15 s), or `pytest --cov=drowsiness` for the coverage table.

8.1 Kinds of tests in this project

Kind	Example	What it proves
Unit test	`test_ear_open_eye`: EAR of a hand-made eye is exactly 0.4	A single function computes the right value
Behaviour/timing test	`test_microsleep_reaches_full_score`: 1.6 s closed gives score 1	Logic over time behaves as designed
Edge-case test	`test_ear_degenerate_eye`: all points equal gives 0, not a crash	No division by zero or crash on odd input
Fake/mock test	`test_lost_camera_raises_after_retrying` with a fake camera	Error handling works without real hardware
End-to-end test	`test_calibrates_then_detects_closed_eyes` with the real model	All modules work together on real images
Property check on real data	`test_head_pose_on_real_face_geometry`	Pose signs are right for a real face, not only the model
UI smoke test	`test_dashboard_renders_with_disclaimer` (Streamlit AppTest)	The dashboard starts without errors

8.2 What the tests do NOT prove

> **Important.** The tests prove the code **does what it was designed to do**. They do not prove the design **detects real drowsiness accurately**. That needs labelled video of real drowsy and alert people and the metrics in section 12. Say this clearly in your report and viva.

- Only two real images are used; real video has motion blur, lighting changes and many faces.
- Thresholds (MAR 0.5, 15° nod, 20° roll, weights, alert levels) are starting values.
- The alarm sound and a physically unplugged camera were not tested live; their logic is unit-tested with fakes.
- `dashboard.py` shows 0% in the coverage table because Streamlit's test harness runs it in a way coverage cannot trace, even though its tests run and pass.

8.3 Live checks done on your laptop

- Webcam pipeline: about 26 FPS, 9.5 ms processing per frame.
- Dashboard listens on 127.0.0.1 only; a second tab is refused; settings lock while running; the camera lock is released after stopping.
- With no face in view: Warning at 3 s, Alarm at 6 s, Critical at 10 s, plus the "no face seen during calibration" note.
- Your own sessions on 26–30 September triggered the full Warning → Alarm → Critical escalation.

9. Engineering practices used

Practice	In this project	Why it matters
Virtual environment	`.venv`	Packages for this project don't clash with others
Pinned dependencies	`constraints.txt`	Same versions everywhere; reproducible results
src layout + pyproject	`src/drowsiness`, `pyproject.toml`	Tests run against the installed package, like real users
Typed, validated config	`config.py`, `default.toml`	One place for every number; typos and bad ranges fail fast
Type hints	every function	Readable signatures; editors and type checkers catch mistakes
Logging, not print	`logging` module	Levels (info/warning/error) and timestamps; easy to silence or redirect
Automated tests + CI	`tests/`, GitHub Actions	Every change is checked on a clean machine
Linting/formatting	ruff	Consistent style; catches common bugs
Branches + pull requests	PR #1, PR #2	Changes reviewed before reaching main
Fail loudly	face loss, camera loss	Safety: never stop protecting silently
Privacy by default	local-only dashboard, no video saved	The webcam stream is sensitive
Honest documentation	README warning and status	Claims match the evidence

9.1 Git commands you will use daily

```
git status                          # what changed
git switch -c my-feature            # new branch for a change
git add -A && git commit -m "Explain what and why"
git push -u origin my-feature       # then open a pull request on GitHub
git switch main && git pull         # get the latest main after a merge
gh pr create                        # open a PR from the terminal (GitHub CLI)
```

10. Safety, privacy and the audit

A full audit (correctness, security, safety, reliability, testing, documentation) was done on 10 October 2026. Everything that could be fixed in code is fixed in PR #2.

ID	Problem found	Fix
F-01	Face lost (head slumped, camera blocked) stayed at a silent warning forever	Escalates to alarm at 6 s and critical at 10 s
F-02	Dashboard reachable from the whole network without login, showing the webcam	Local-only by default (127.0.0.1)
F-03	New code was not committed anywhere	Committed; PRs #1 and #2
F-04	A locked log file or a camera glitch stopped monitoring silently	Log retries; camera retry/reopen; alarm until acknowledged
F-05	Calibrating with closed eyes was accepted	Plausibility check, restart, visible fallback
F-06	Eyes alone could not exceed Level 2 via the score	Microsleep always raises at least Level 2
F-08	Two dashboard tabs ran two loops on one camera	One session at a time; settings locked while running
F-09	One noisy frame reset the microsleep timer	0.15 s debounce
F-11	Zero/negative settings crashed the program	Range validation
F-12	Video timing assumed a constant frame rate	Uses the file's own timestamps
F-07	"PERCLOS" name over-claimed; thresholds untuned	Renamed PERCLOS-like; tuning needs Phase 4 data

10.1 What is still not safe

- Accuracy is unmeasured; the system can miss drowsiness or raise false alarms.
- Very dark conditions, infrared-only night driving, sunglasses, masks and extreme head angles are untested.
- A laptop app has no protection if the computer sleeps, the battery dies or the process is killed.
- Alert logs reveal when a person appeared drowsy: treat them as personal data and delete them when no longer needed.

11. Other ways to build a drowsiness detector

Your project uses **hand-designed geometric features from landmarks** (EAR, MAR, head pose) plus rules. It is explainable, needs no training data and runs on a laptop CPU. Know the alternatives and their trade-offs:

Approach	How it works	Pros	Cons
Landmark geometry + rules (this project)	EAR/MAR/pose from facial points; thresholds over time	No training; explainable; fast; works per driver via calibration	Thresholds need tuning; sensitive to landmark errors (glasses, dark)
Eye-state CNN	A small neural network classifies eye crops as open/closed (public sets such as MRL Eye or Closed Eyes in the Wild)	Learns appearance; robust to some lighting	Needs labelled data and training; less explainable
Video deep learning	3D CNNs or CNN + LSTM on clips (datasets such as NTHU-DDD, YawDD)	Captures temporal patterns end to end	Large data, GPU training, risk of overfitting to actors
Blendshape-based	Use MediaPipe's learned `eyeBlink`/`jawOpen` scores instead of geometry	Already available; robust to face shape	Black box; still needs thresholds
Infrared camera	Near-IR illumination + IR camera (common in commercial cars)	Works at night and through many sunglasses	Special hardware
Vehicle-based	Steering movements, lane position, speed variation	No camera; privacy friendly	Needs vehicle data; reacts late
Physiological	EEG, ECG/heart-rate variability, skin conductance	Most direct measure of fatigue	Wearable sensors; intrusive

A strong report says why you chose landmark geometry (proposal scope, CPU-only, explainable, per-driver calibration) and names one or two of these as future work.

12. What is left: Phase 3 and Phase 4

12.1 Phase 3 — condition detection and adaptive fusion (CE-2, Nov 1)

Goal: detect when the eye signal becomes unreliable (low light, glasses) and shift weight to cues that still work.

- **Low light:** measure brightness of the face region (mean grey level, histogram) and MediaPipe's confidence. A synthetic test already showed that darkening the image shrinks the open/closed EAR gap (open 0.328 → 0.282, closed 0.112 → 0.202 at 5% brightness) before the face is lost.
- **Glasses:** options include edge density on the nose bridge between the eyes (frames create strong edges), unusually noisy EAR, and disagreement between geometric EAR and the `eyeBlink` blendshapes.
- **Adaptive fusion:** a new `AdaptiveFusion` class implementing the `FusionStrategy` protocol, with a weight table per condition, for example:

Condition	Eyes	Yawn	Head
Normal (= static weights)	0.50	0.20	0.30
Low light	0.20	0.35	0.45
Eye-region occlusion (glasses)	0.20	0.35	0.45

Same example values as the midsem report. Normal equals the static weights, so any measured difference comes only from adapting to conditions. They are a starting point, not tuned values.

12.2 Phase 4 — evaluation

- **Data:** request NTHU-DDD and YawDD early (access takes time). Record your own clips: glasses on/off × day/night × alert/acting drowsy.
- **Labels:** for each clip, a CSV of drowsy segments (start_s, end_s).
- **Run:** `drowsiness process clip.mp4` gives per-frame values; run static and adaptive fusion on the same frames.
- **Split:** tune thresholds and weights on some clips, report results on *different* clips; otherwise the results are over-optimistic.
- **Metrics:** precision, recall (sensitivity), F1, false-negative rate under adverse conditions (your main claim), detection latency (seconds from drowsy onset to alarm), false alarms per hour, FPS.

```
precision = TP / (TP + FP)      of the alarms raised, how many were real drowsiness
recall    = TP / (TP + FN)      of the real drowsy episodes, how many were caught
F1        = 2 * precision * recall / (precision + recall)
```

13. What to learn (in order)

A realistic roadmap from basic Python to building and evaluating this system yourself. Times assume a few hours per day.

Step	Learn	Why for this project	Practice
1 (1–2 wk)	Python: functions, classes, dataclasses, exceptions, `with`, type hints, modules/packages	Everything is Python	Rewrite `eye_aspect_ratio` and a tiny `Calibrator` yourself
2 (1 wk)	NumPy: arrays, indexing, broadcasting, `linalg.norm`	Landmarks are arrays; all geometry	Compute EAR for 478 random points with index lists
3 (1–2 wk)	OpenCV: read webcam/video, BGR vs RGB, resize, flip, draw, `imshow/waitKey`	Camera input and overlay	Show your webcam with your FPS on screen
4 (1 wk)	Geometry: vectors, distances, rotation matrices, Euler angles, pinhole camera	EAR/MAR, head pose	Project a 3D cube onto an image with `projectPoints`
5 (1 wk)	MediaPipe Tasks FaceLandmarker (official docs)	The landmark source	Draw the 478 points with their index numbers
6 (1 wk)	Time series: sliding windows, debouncing, hysteresis, state machines	Indicators and alerts	Build a blink counter that ignores single noisy frames
7 (3–4 days)	Streamlit: widgets, re-runs, `session_state`, `cache_resource`	Dashboard	A page with a live webcam and a slider
8 (3–4 days)	pytest, fixtures, monkeypatch; ruff	Confidence in changes	Write tests for your blink counter
9 (3–4 days)	Git, branches, pull requests, GitHub Actions	Collaboration and CI	Open a PR in a practice repo with a CI workflow
10 (1 wk)	Evaluation: confusion matrix, precision/recall/F1, train/test split, data leakage	Phase 4	Evaluate your blink counter on labelled clips
Optional	Deep learning basics (PyTorch), CNNs for eye state	Section 11 alternatives	Train an open/closed eye classifier

13.1 Reliable places to learn from

- Python tutorial and reference: docs.python.org/3/tutorial
- NumPy: numpy.org/doc (start with "NumPy: the absolute basics for beginners")
- OpenCV Python tutorials: docs.opencv.org ("OpenCV-Python Tutorials"; camera calibration and pose estimation chapters for solvePnP)
- MediaPipe Face Landmarker guide for Python: ai.google.dev/edge/mediapipe
- Streamlit documentation: docs.streamlit.io
- pytest documentation: docs.pytest.org
- Pro Git book (free): git-scm.com/book
- Metrics: scikit-learn.org, user guide section on model evaluation
- Soukupová and Čech (2016), "Real-Time Eye Blink Detection using Facial Landmarks" (the EAR paper)

14. Build it yourself: a milestone plan

If you rebuilt this from an empty folder, this order keeps something working at every step:

#	Milestone	Done when
1	Webcam window with OpenCV	You see yourself and the FPS
2	MediaPipe landmarks drawn on the face	478 dots follow your face
3	EAR and MAR printed live	EAR drops when you close your eyes
4	Blink vs long-closure detection with timestamps	Blinks ignored; 1.5 s closure flagged
5	Alarm sound with pygame	Sound starts/stops with the closure
6	Calibration (median over 5 s)	Threshold adapts to a second person
7	Head pose with solvePnP	Pitch rises when you nod
8	Yawn and head indicators + weighted fusion	One score 0–1 combines the three
9	Alert state machine with hysteresis/escalation	Levels change cleanly, no flicker
10	Config file + validation	All numbers live in one TOML file
11	Tests for each piece	`pytest` green
12	Streamlit dashboard	Live view, metrics, chart, history
13	Packaging, CLI, CI	`pip install -e .` gives a `drowsiness` command; GitHub runs tests
14	Evaluation on labelled clips	You can report precision/recall honestly

15. Viva and review questions (with answers)

Question	Short answer
Why EAR instead of training a CNN?	No training data needed, explainable, fast on CPU, adapts per driver via calibration. A CNN is listed as future work.
Why MediaPipe instead of dlib?	478 vs 68 points, better with glasses and pose, 3.6 MB vs 95 MB model, easy install, named in the proposal.
Why does EAR not reach 0 when eyes close?	Landmarks never place the lids exactly together; closed eyes measured about 0.11.
Why calibrate?	Open-eye EAR differs between people and with glasses/camera position; a fixed threshold misfires.
Why the median during calibration?	It ignores blinks and outliers; the mean would be pulled down by blinks.
How do you tell a blink from drowsiness?	Duration and frequency over time: blinks last about 0.1–0.4 s; a closure of 1.5 s or more is a microsleep; the closed-eye ratio captures frequent closures.
Is your metric PERCLOS?	PERCLOS-like: share of time eyes are closed, but with a looser closure criterion (EAR below 75% of open) and a 20 s window, not P80 over minutes.
How is head pose computed?	solvePnP fits a generic 3D face model's six points to the detected landmarks; rotation decomposed into pitch/yaw/roll; relative to the calibrated pose.
Why a weighted average, and what is its weakness?	Simple, interpretable baseline for comparison; a single cue has a ceiling, so a microsleep rule guarantees an alarm.
Why hysteresis?	Prevents rapid on/off switching when the score sits near a threshold.
What happens if the face disappears?	Warning at 3 s, alarm at 6 s, critical at 10 s: a slumped head must not be silent.
How do you know it works?	96 tests prove the design behaves as intended; real accuracy is Phase 4 with labelled data and precision/recall/latency.
Why is the dashboard local-only?	It streams the webcam without a login; exposing it on a network is a privacy risk.
What runs in real time?	Full pipeline about 9.5 ms per frame on a laptop CPU; the webcam limits it to about 26 FPS.
What would you do with more time?	Adaptive fusion (Phase 3), evaluation (Phase 4), IR camera for night, eye-state CNN as a second opinion.

16. Glossary

Term	Meaning
Landmark	A numbered point at a fixed place on the face
EAR	Eye Aspect Ratio: eye height ÷ width from six points
MAR	Mouth Aspect Ratio: inner-lip opening ÷ mouth width
Pitch / yaw / roll	Head rotation: nodding / turning left-right / tilting sideways
solvePnP	OpenCV function finding an object's pose from known 3D points and their 2D image positions
PERCLOS	Percentage of eyelid closure over time (classically ≥80% closed)
Microsleep	A short involuntary sleep episode; here, eyes closed ≥1.5 s
Calibration	Learning a driver's personal baseline at the start
Fusion	Combining several cues into one score
Hysteresis	Different thresholds for going up and coming down, to avoid flicker
Debounce	Ignoring a change until it has lasted a minimum time
State machine	A system with defined states and rules for moving between them
Blendshape	MediaPipe's learned 0–1 score for a facial movement (e.g. eyeBlinkLeft)
FPS	Frames per second
Virtual environment	A private folder of packages for one project
CI	Continuous Integration: tests run automatically on every change
Pull request	A request to merge a branch, where changes are reviewed
Precision / recall	Share of alarms that were correct / share of real events that were caught
False negative	Real drowsiness that the system missed: the dangerous error

> **Note.** This guide describes the code on branch `fix/audit-findings` (PR #2). If the code changes later, the README and `configs/default.toml` are the up-to-date references.